Replication of Two-Lane Traffic Simulations using Cellular Automata

IMS Project T8: Cellular Automata in traffic

5. December 2025

Lukáš Eliáš, xeliasl00
Jacek Folwarczny, xfolwaj00

Contents

1 Introduction

This report presents a replication of the two-lane traffic cellular automata model introduced by Rickert et al. (1996) [1]. The study aims to verify the reported macroscopic flow characteristics and microscopic lane-changing behaviors. We successfully reproduced the fundamental flow-density diagrams and spatio-temporal traffic jam dynamics. We demonstrate that the high lane-change rates reported in the original study are influenced by a specific evaluation of lane-change rules. This report details the implementation methodology, validates the macroscopic results, and provides insight into the behavior of microscopic artifacts by examining data acquired through experiments with a cellular automata model.

The seminal work by Rickert et al. [1] extended the single-lane Nagel-Schreckenberg model [2] to two lanes. The primary objective of this work is to validate the results of the 1996 paper by strictly implementing the rulesets described in the text and replicating the key data visualizations. We anticipate reproducing the general shape of the flow rate graphs (Figures 3 and 4 in the original text), the behavior of traffic jams over time (Figures 1,2,12, and 13 in the original text), and the impact of experimenting with other parameter combinations (Figures 7 and 11 in the original text).A secondary objective is to present the effect of different update rule interpretations from the original paper by comparing the values of lane-changing frequency for two models that differ in specific implementation details regarding sensor boundaries.

2 Review of the Reference Model

The reference paper, "Two lane traffic simulations using cellular automata" [1], proposes a minimal set of rules to govern lane changing in a discrete system. The model builds upon the single-lane rules of acceleration, deceleration, randomization, and movement[2].

1. **Acceleration (S1):** Increase speed if not at maximum velocity.
   (**IF** $v(i) \neq v_{max}(i)$ **THEN** $v(i) := v(i) + 1$)
2. **Deceleration (S2):** Reduce speed to avoid collision with the predecessor.
   (**IF** $v(i) > gap(i)$ **THEN** $v(i) := gap(i)$)
3. **Randomization (S3):** Stochastic braking with probability p_d.
   (**IF** $v(i) > 0$ **AND** $rand < p_d(i)$ **THEN** $v(i) := v(i) - 1$)

The gap to the preceding vehicle is defined as follows:
Let $x_n(t)$ and $d_n(t)$ denote the position and ahead gap of the n-th vehicle, then:

$$d_n(t) = x_{n+1}(t) - x_n(t) - 1 \tag{1}$$

Note that if there is a vehicle on the neighboring site, the gap is equal to the value -1.

The core contribution is the introduction of a generic lane-changing ruleset defined by four criteria:

1. **Incentive (T1):** Is the driver blocked in their current lane? ($gap < l$)
2. **Improvement (T2):** Is the target lane better? ($gap_o > l_o$)
3. **Safety (T3):** Is the target lane switch safe? ($gap_{o,back} > l_{o,back}$)
4. **Probability (T4):** A stochastic parameter p_{change} denoting lane switch probability.
   ($rand < p_{change}$)

The parameters l, l_o, and $l_{o,back}$ determine how far the vehicle looks ahead in its current lane, the other lane, or back in the other lane, respectively. Note the use of strict inequalities. The original paper [1] uses values $l = v + 1$, $l_o = l$, $l_{o,back} = 5 = v_{max}$, and $p_{change} = 1$, unless stated otherwise.

The paper also explores two variants: a *Symmetric* model, where drivers only change lanes to evade an obstacle in front of them, and an *Asymmetric* model, where drivers obey a "keep right" rule similar to how real-life drivers are accustomed to act in many cultures. For the model itself, the Symmetric variant requires all rules from the ruleset to be fulfilled for the vehicle to switch to the other lane at all times. The Asymmetric variant vacates the fulfillment of the Incentive (T1) rule when a vehicle wants to switch from the right lane to the left.

3 Model Conception and Evaluation Metrics

3.1 System Definition

The simulation is defined on a lattice of two parallel lanes of length L, representing roads. We utilize periodic boundary conditions, topologically representing a ring road. The state of the system at time t is updated to $t + 1$ using discrete time steps.

The update dynamics of one time step follows a strict **Split-Step Parallel Update** scheme:

1. **Lane Change Step:** Vehicles evaluate rules T1–T4 based on the configuration at time t. Vehicles satisfying the conditions move laterally to the adjacent lane; the vehicles do not advance.
2. **Velocity and Motion Step:** Vehicles update their velocities (rules S1–S3) based on the new configuration after the Lane Change step and move forward accordingly.

This split-step architecture is critical. It implies that a vehicle's decision to brake (velocity update) is influenced by its new neighbors immediately after changing lanes, creating a coherent update mechanism.

3.2 Metrics

Two primary metrics are utilized to evaluate traffic dynamics: the macroscopic flow and the microscopic lane-changing frequency. Flow is averaged over space (system size L) and time (duration T). Lane-changing frequency is averaged per site on the road over time (Figure 9) or per vehicle over time (Figure 10).

3.2.1 Flow

The flow j represents the throughput of the system and is calculated as the space-time average of the velocities of all vehicles. In the context of the two-lane cellular automaton, the flow is defined as:

$$\langle j \rangle_{L,T} = \frac{1}{N_T \cdot L} \sum_{t=1}^{N_T} \sum_{i=1}^{L} v(i,t) \tag{2}$$

where:

- L is the number of sites in the system.
- N_T is the number of sampling time steps.
- $v(i,t)$ is the velocity of the vehicle at site i at time t (where $v = 0$ if the site is empty).

To avoid correlation effects between consecutive system states, the simulation statistics for flow are gathered only every fifth time step (i.e., sampling interval $\Delta t = 5$). Consequently, the summation over time covers $T/5$ discrete measurements rather than every single time step.

3.2.2 Lane Change Frequency

The lane change frequency measures the imperfections in traffic flow. It is defined as the number of lane changes per site per time step. Unlike the flow metric, lane changing statistics are gathered at every time step to capture all lateral movements. The calculation is given by:

$$f_{LC} = \frac{N_{changes}}{L_n \cdot T_{total}} \tag{3}$$

where:

- $N_{changes}$ is the total count of lane changes observed during the measurement period.
- L_n is the total number of neighboring sites in the system.
- T_{total} is the total number of simulated time steps.

The Lane Change Frequency per Car metric is then calculated by multiplying f_{LC} by the car density. This metric helps to reveal the usage of the introduced lane change mechanic for different model configurations.

4 Implementation

The core microscopic traffic model and time-stepping simulation are implemented in C++, complemented by a Bash script for experiment automation and a Python notebook for visualization. The C++ codebase encapsulates the model dynamics, state evolution, and data export. The Bash script orchestrates repeated runs with different parameter configurations and directs the outputs into a structured data directory. A Python Jupyter notebook then loads the generated CSV files and produces plots of flow and car change frequency diagrams.

4.1 Codebase Structure

The main simulation logic resides in the `src/` directory. The entry point `main.cpp` parses command-line arguments, initializes the simulation, and triggers the experiment run. The file `simulation.cpp` (with the interface `simulation.hpp`) implements the core update loop over discrete time steps and manages interactions between roads and lanes, leveraging a Double-Buffering architecture for correct parallel update handling. The `state.cpp` and `state.hpp` files define the data structures representing the current traffic configuration, including vehicles, lanes, and positions. Vehicle dynamics such as acceleration and deceleration are encapsulated in `velocity.cpp` and `velocity.hpp`, while lane changing logic (e.g., gap checking and safety criteria) is implemented in `lane_change.cpp` and `lane_change.hpp`. The file `data_gathering.cpp` (with `data_gathering.hpp`) handles the collection and aggregation of observables, writing them to CSV files. Common type definitions and utility structures are centralized in `types.hpp`.

Experiment automation is performed by the `run.sh` script in the project root, which runs batches of simulations with different parameter sets and stores the results in the `csv/` folder. The Python notebook `data/plots/graphs/visualize_data.ipynb` loads these CSV files and generates plots such as flow-density diagrams and speed profiles, which are then saved into the `data/plots/` subdirectories.

4.2 Execution and Run Arguments

The C++ binary can be built using a Makefile; the resulting executable is named $ims\_traffic$ and accepts multiple command-line arguments to configure a simulation run. Typical parameters include the system size (number of cells per lane), the simulation time horizon, the global vehicle density, and model-specific parameters such as maximum speed, symmetry/asymmetry, slow-down probability, and lane-change probability. Additional flags control output behavior, such as metric output in CSV format or the generation of a time-space evolution bitmap of the traffic.

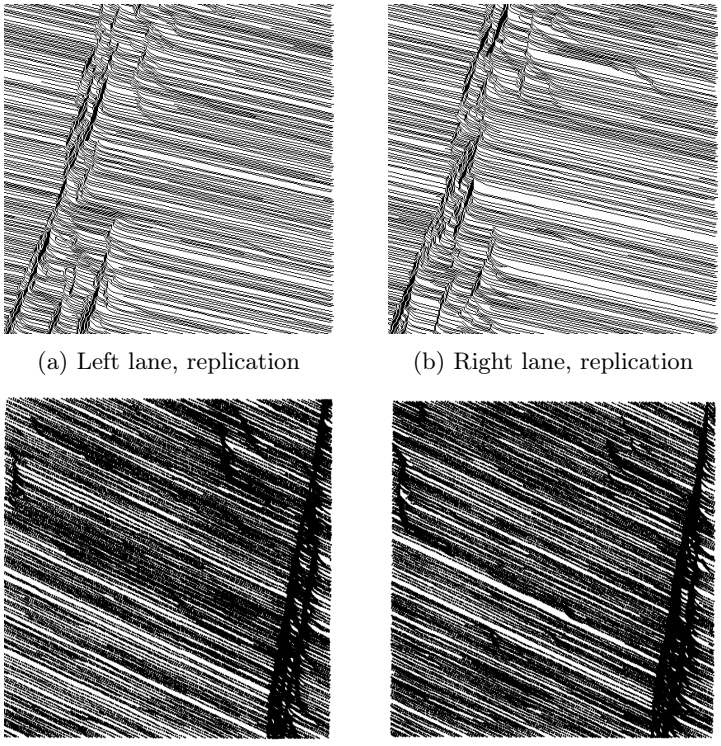
The `run.sh` script exposes a higher-level interface to these arguments by defining experiment presets. Each preset corresponds to a specific combination of model type (e.g., symmetric vs. asymmetric), parameter values, and output filenames. By invoking the script with a small set of options (for example, to choose the model variant or the density grid), users can automatically launch a series of simulations and collect all resulting data in a reproducible and organized manner, ready for subsequent analysis in the Python notebook.

5 Experimental Results

This section presents the comparison of the outputs provided in the original paper and the outputs generated by our replication model. The experiments are divided into subsections, each focused on different aspects of the model. For every experiment, the system ran for 1000 steps before gathering any data. This approach allows the system to settle transient issues caused by initial conditions.

5.1 Spatio-Temporal Dynamics

We first qualitatively verified the model by reproducing and comparing the space-time plots with a length of 12,000 sites, from which we plotted 400 sites in 400 consecutive time-steps. The density is 0.09, which is slightly above the maximum flow density (see Figure 5). Vehicles go from left to right (spatial axis) and from top to bottom (time axis). Omitting the difference in visualization method, both the original and replication traffic jams in Figures 1 and 2 appear as solid areas with steep positive inclination whereas free flow areas are light and have a more shallow negative inclination[1]. Each plot is split into parts: the left part containing the left lane, the right part containing the right lane, the top row presenting the results from our model, and the bottom row showing the original results.



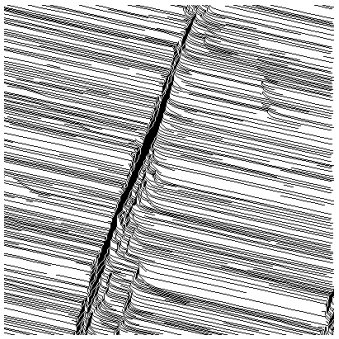
(a) Left lane, replication

(b) Right lane, replication

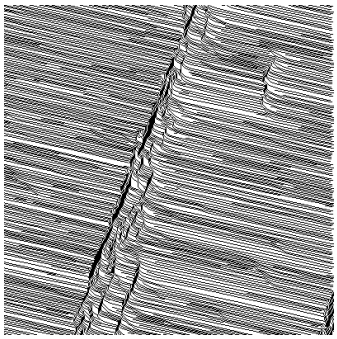
(c) Left lane, original

(d) Right lane, original

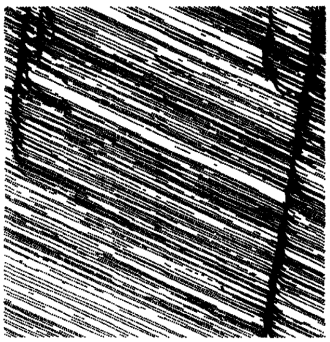
Figure 1: Symmetric, $l_{o,back} = 5$, left lane + right lane, replication x original



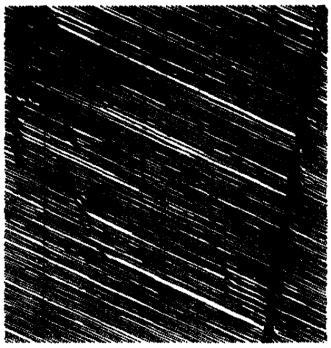
(a) Left lane, replication



(b) Right lane, replication

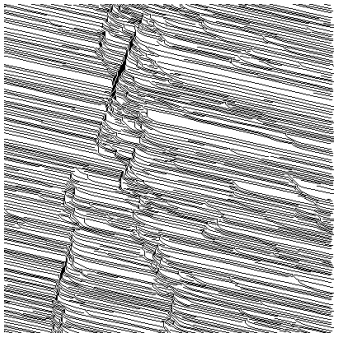


(c) Left lane, original

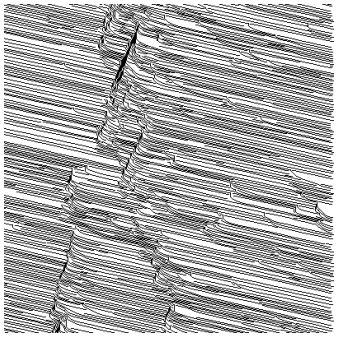


(d) Right lane, original

Figure 2: Asymmetric, $l_{o,back} = 5$



(a) Left lane, replication

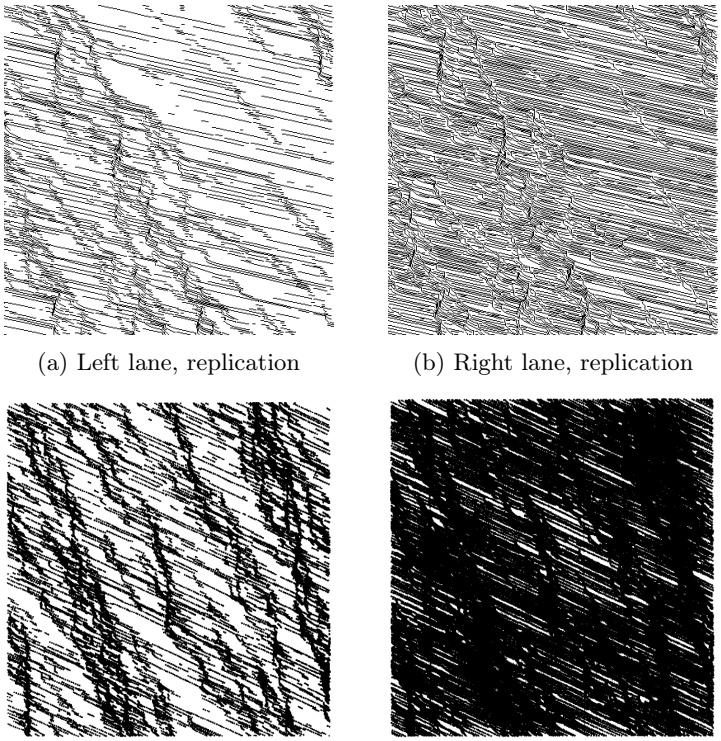


(b) Right lane, replication

(c) Left lane, original

(d) Right lane, original

Figure 3: Symmetric, $l_{o,back} = 0$



(a) Left lane, replication (b) Right lane, replication

(c) Left lane, original (d) Right lane, original

Figure 4: Asymmetric, $l_{o,back} = 0$

The simulation successfully reproduces the breakdown of laminar flow in both the asymmetric and symmetric cases when $l_{o,back} = 0$ (Figures 3 and 4). The aggressive cut-offs create backward-moving shockwaves that match the visual texture of Rickert's Figures 12 and 13.

5.2 Flow Diagrams

Macroscopic flow validation was performed by varying density $\rho \in [0, 0.5]$. The original experiments were conducted for 5,000 steps and a lane length of 133,333 sites, with a density step of 0.01. Due to the limits of our computational resources, we lowered the lane length to 20,000 sites and doubled the density step to 0.02. The graphs presenting our output data are thus more sparse; however, the overall direction of the output values is perceivable.

The upcoming Figures 5-10 show density to flow ratios for different model parameter settings. The upper graphs present outputs from our model, while the bottom graphs show the original figures presented in [1].

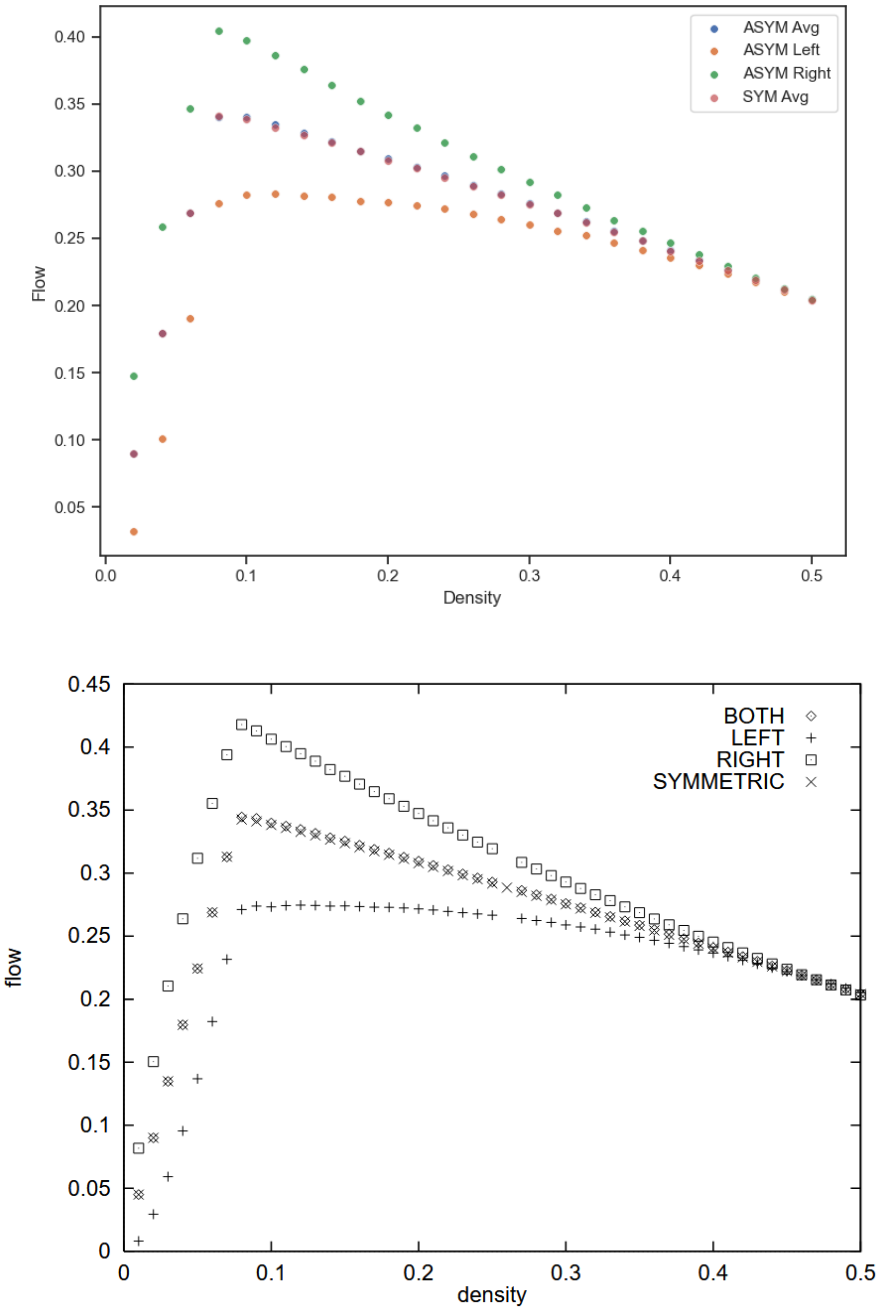


Figure 5: Flow over density, Symmetric and Asymmetric approach, replication - original results

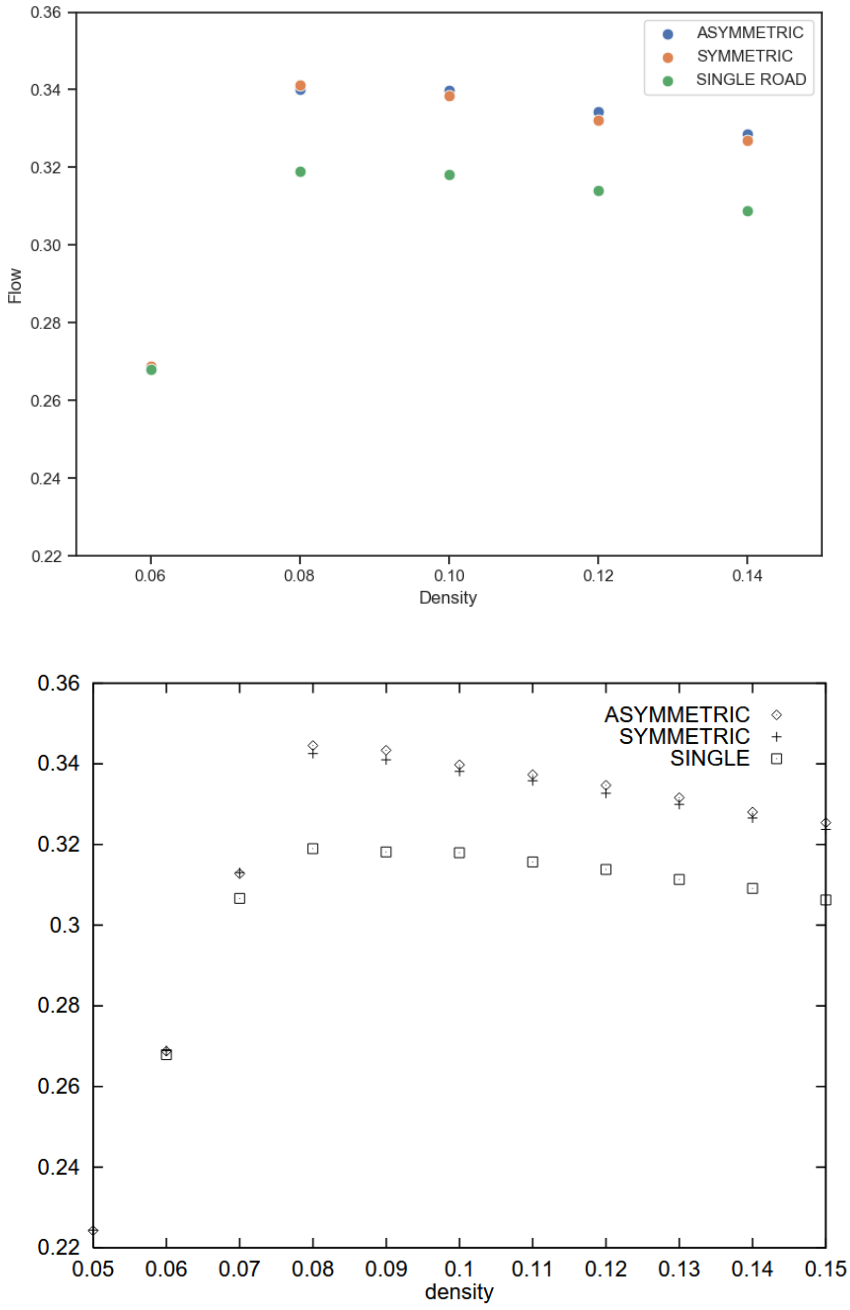


Figure 6: Comparison Single Lane to Multi Lane, replication - original results

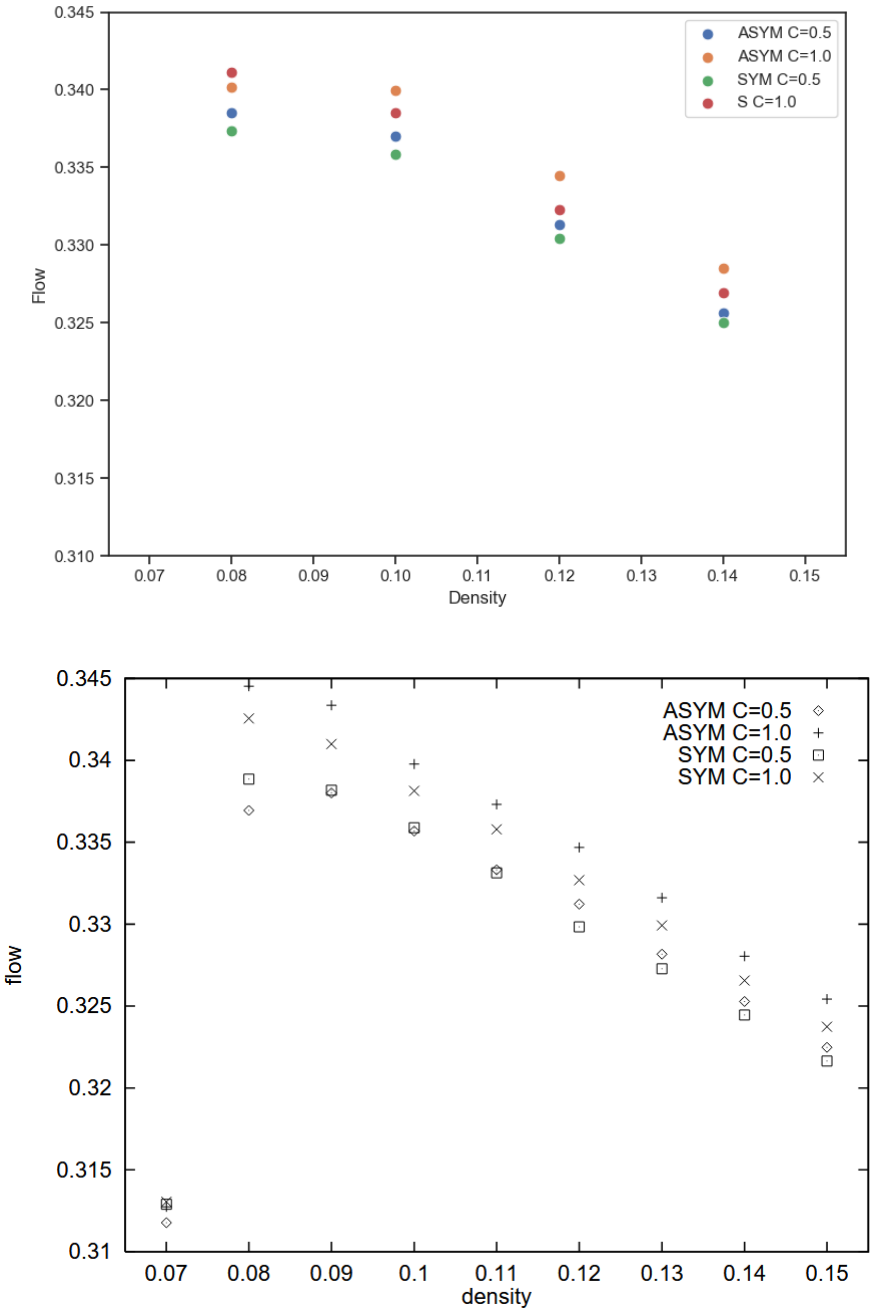


Figure 7: Flow LookBack = 5, different p_{change}, replication - original results

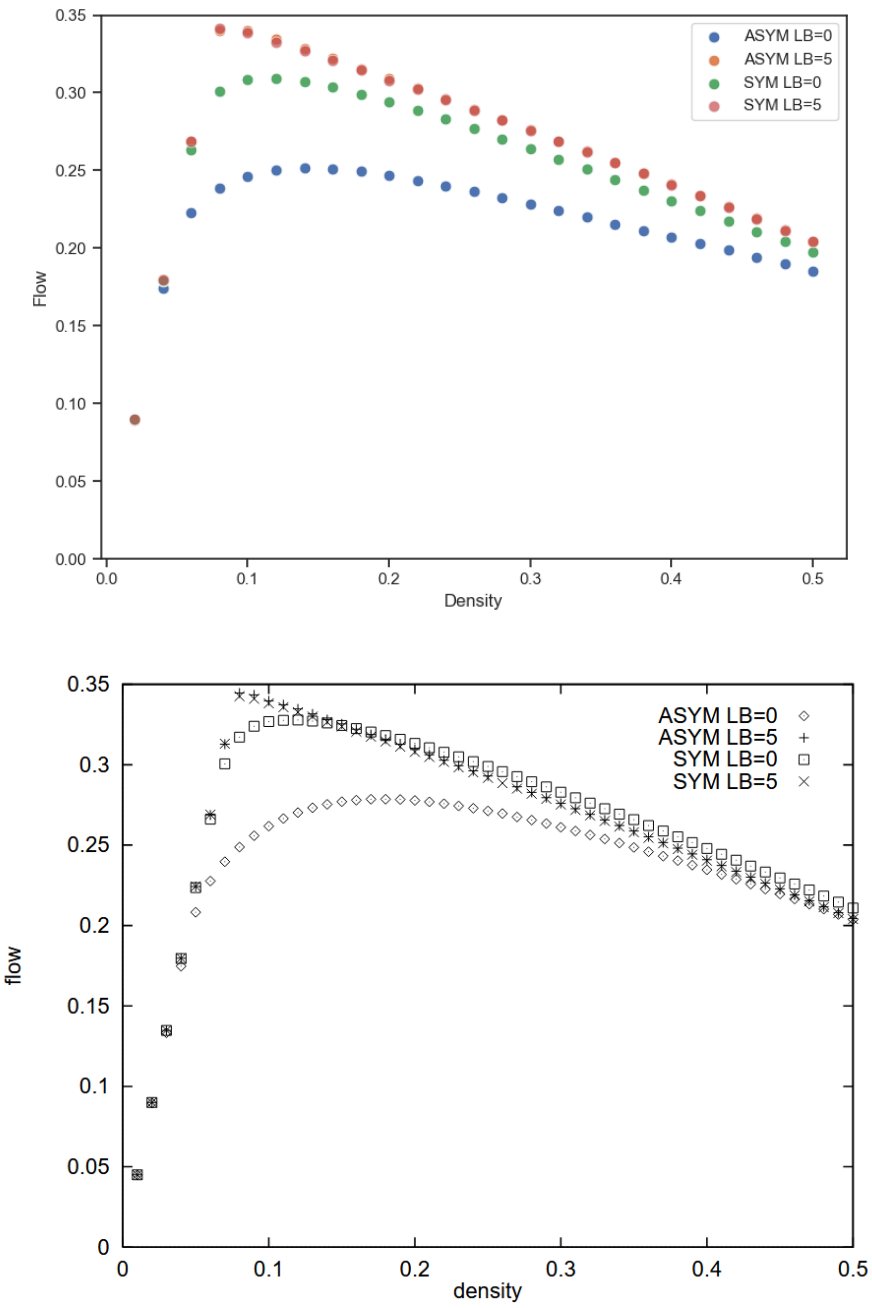


Figure 8: Flow $p_{change} = 1.0$, different Look Back, replication - original results

5.3 Lane Changing Frequency

This section addresses the primary discrepancy found during replication. Figure 5 of the original paper (here Figure 9C) reports a lane change frequency peaking at ≈ 0.0075 changes per site per tick.

5.3.1 Initial Strict Implementation

Our initial implementation evaluated the lane-change rules (T1-T3) with strict inequalities, as stated in section 2 of this report. The evaluation of the model yielded a peak lane change frequency of ≈ 0.003. Additionally, the original paper stated that for the asymmetric case, there is a sharp bend in the curve at density ≈ 0.08 that is almost absent in the resulting graph (see Figure 9A).

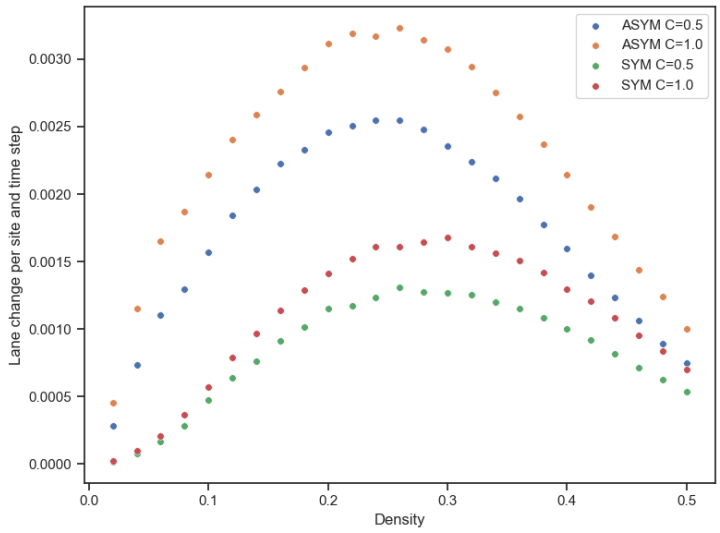
5.3.2 Relaxed Implementation

We experimented with changing the strict inequalities on lane-change rules (T1-T3) to include equality. This relaxed ruleset still prevents vehicles from colliding by the definition of the gap calculation. This equality involving rules dramatically changed the average lane change per site and time step values (from a peak at ≈ 0.003 to ≈ 0.008) when compared with strict implementation(see Figures 9B and 9A). Also, the sharp bend curve mentioned now appears much more apparent.

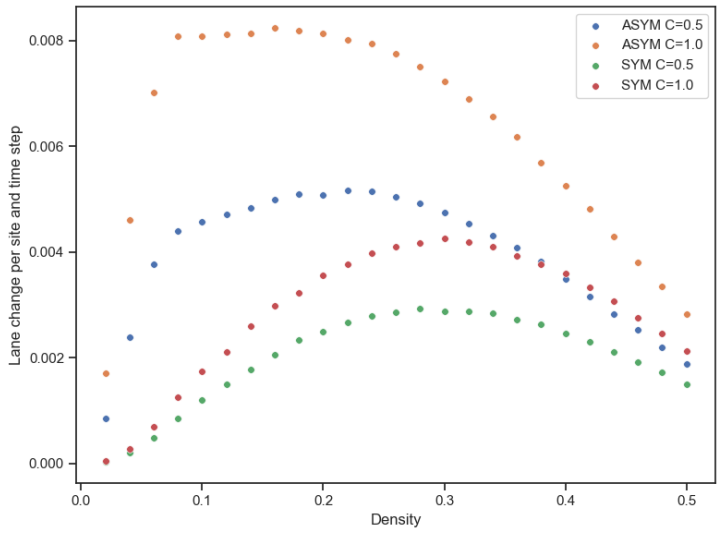
5.3.3 The Artifact

The original paper identifies that the high lane change frequency is an algorithm induced artifact presented as the "Tailgating Dance" or "Ping-Pong" effect. This artifact requires specific conditions to sustain a lane changing resonance where a vehicle Switches Left $\rightarrow$ Brakes $\rightarrow$ Switches Right $\rightarrow$ Accelerates $\rightarrow$ Switches Left, generating a significant number of redundant lane switches that do not comply with the behavior of a real driver.

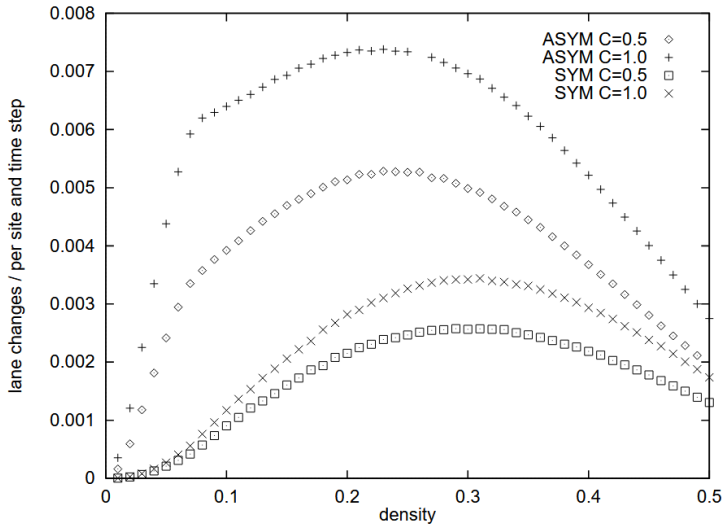
The state of the simulation and the sequence of specific actions guided by stochasticity required for the artifact to appear are less likely in the Strict Implementation than in the Relaxed Implementation, explaining the discrepancies in the observed values. By examining all graphs from Figure 9 and Figure 10, it is noticeable that neither the Strict nor the Relaxed ruleset perfectly matches the original values and curves presented, indicating other possible implementation options. It is interesting to point out that even though the lane change frequency changes significantly with differing rulesets, the flow to density graphs were hardly differentiable between the implementations. Replication results in Figures 5 to 8 were conducted on a model using the Relaxed ruleset. Data outputs and graphs from experiments with the Strict Implementation can be found within the archive submitted along with this report.



(a) Strict ruleset

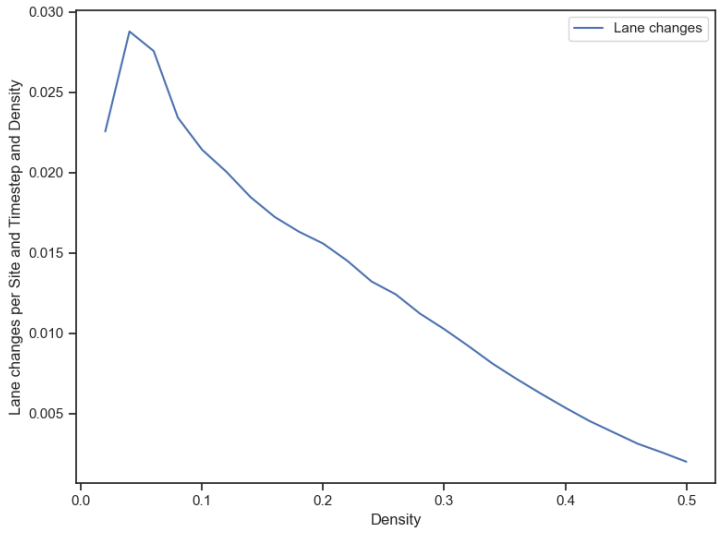


(b) Relaxed ruleset

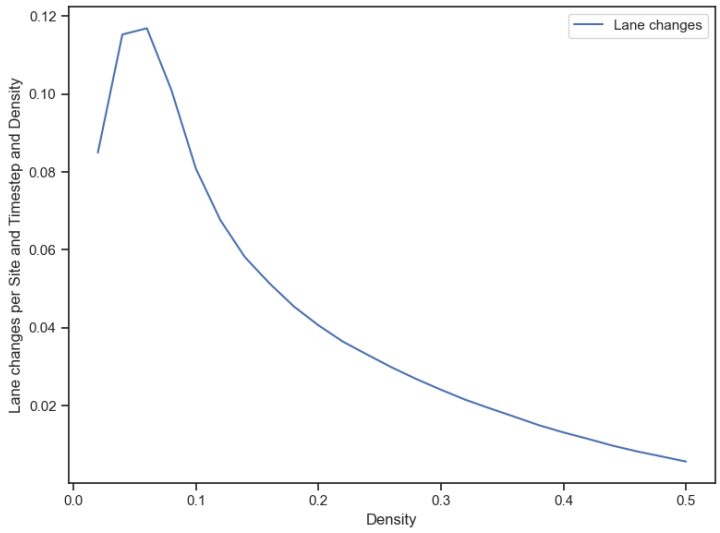


(c) Original

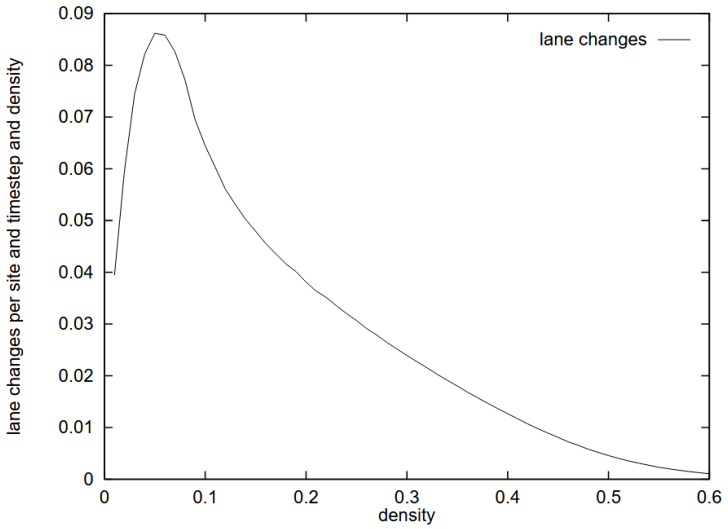
Figure 9: Simple Lane Changes per Site and Time with different approaches and p_{change}



(a) Strict ruleset



(b) Relaxed ruleset



(c) Original

Figure 10: Lane changes per vehicle, $p_{change} = 1$

6 Summary and Conclusion

This study successfully replicated the two-lane cellular automata model proposed by Rickert et al. (1996) [1].

- **Macroscopic Validation:** The fundamental diagrams for flow and density are robust and reproducible. The model correctly predicts that lane changing increases overall system flow rate.
- **Microscopic Discrepancy:** The lane-changing frequency reported in the original paper is highly sensitive to implementation details. We support the original findings that the high frequency (≈ 0.0075) was caused by the specific evaluation of the update order and gap checks.
- **Scientific Consensus:** The artifact observed in the model's behavior aligns with later literature [3], which characterizes the high lane-changing frequency as unrealistic ping-pong behavior.

In conclusion, while the macroscopic conclusions of the 1996 paper hold, the microscopic lane-changing statistics should be interpreted with caution, as they are heavily influenced by model artifacts caused by specific implementation details. The original study presents an expanded investigation of the artifact behavior by thoroughly tracking different kinds of the Ping-Pong effect. We have not covered this research area in our implementation, leaving it as an expansion for future work.

References

[1] Rickert, M., Nagel, K., Schreckenberg, M., & Latour, A. (1996). *Two lane traffic simulations using cellular automata.* Physica A: Statistical Mechanics and its Applications, 231(4), 534-550.

[2] Nagel, K., & Schreckenberg, M. (1992). A cellular automaton model for freeway traffic. Journal de Physique I, 2(12), 2221-2229.

[3] W Knospe et al 2002. A realistic two-lane traffic model for highway traffic. J. Phys. A: Math. Gen. 35 3369